Maps Says Yes, the Game Says No: Why Non-Jailbreak iOS GPS Spoofing Cannot Beat Modern Anti-Cheat

A coursework research paper documenting the design, implementation, debugging, and ultimate anti-cheat encounter of a host-side iOS GPS spoofing tool (Phantom), and the current (2026) state of Pokemon GO location anti-cheat.

Karan Kantaria — Independent Researcher

June 2026

Contents

1. Abstract . . . 3
2. Introduction & motivation . . . 4
3. Background — how non-jailbreak iOS location spoofing works . . . 4
   - 3.1 Location *provenance*: the concept the whole paper turns on . . . 5
4. Threat model . . . 5
5. Methodology . . . 6
   - 5.1 Apparatus . . . 6
   - 5.2 Procedure . . . 6
   - 5.3 The controlled (differential) test — the core of the method . . . 6
   - 5.4 Planned discriminating experiment — the "real-location" test . . . 7
6. Implementation — what was built ("Phantom") . . . 8
7. Engineering findings — every problem encountered and its fix . . . 8
   - 7.1 Dependency compilation failure . . . 8
   - 7.2 No usbmux / "Failed to connect to usbmuxd socket" . . . 8
   - 7.3 Tunnel hangs (userspace) on Windows . . . 9
   - 7.4 CLI crash when launched elevated: `sys.__stdout__` is None . . . 9
   - 7.5 Admin requirement even for "userspace" on Windows . . . 9
   - 7.6 Blind to elevated processes . . . 9
   - 7.7 Developer Mode would not appear (the biggest onboarding hurdle) . . . 9
   - 7.8 Proof of life . . . 9
8. Results . . . 10
   - 8.1 General spoof: success . . . 10
   - 8.2 The differential test: PoGo alone fails . . . 10
   - 8.3 What was tried before concluding it was anti-cheat (negative controls) . . . 10
   - 8.4 The discriminating ("real-location") trial: coherent fix, still rejected . . . 10
9. Analysis — the trust gate, and a debunked easy answer . . . 12
   - 9.1 A tempting wrong answer: `CLLocationSourceInformation` . . . 12
   - 9.2 Which check fires? Three live hypotheses, ranked by what a sandboxed app can actually observe . . . 13
   - 9.3 Why our method fails: the two classes of no-jailbreak spoofing . . . 15
   - 9.4 Why a laptop's own Bluetooth radio cannot impersonate class B . . . 16
   - 9.5 The software tools are themselves broken on current iOS . . . 17
10. Ethics & legal considerations . . . 17
11. Limitations & future work . . . 18
12. Conclusion & recommendations . . . 19
13. References . . . 19

Platform under test: Windows 11 host · iPhone 11 (`iPhone12,1`) · iOS 26.5 (build 23F77) · Python 3.14 · `pymobiledevice3` 9.30.1

Result in one line: GPS spoofing succeeds system-wide (Apple Maps, Snapchat honor it) but **Pokémon GO rejects it with "Failed to detect location (12)"** — a deliberate anti-cheat block that the software/developer-mode method cannot overcome.

---

1. Abstract

This paper investigates whether a desktop application can feed a fake GPS location to a non-jailbroken iPhone strongly enough to play Pokémon GO from a fixed location. A working host-side spoofer was built on Apple's own developer location-simulation service (`pymobiledevice3`), including the mandatory keep-alive loop and GPS jitter. The spoof works perfectly for ordinary apps but is **specifically detected and blocked by Pokémon GO** ("error 12"). A controlled differential test (Apple Maps and Snapchat both accept the spoof while PoGo alone fails) isolates the cause to Niantic's anti-cheat rather than any implementation defect. We then examine *why*: the obvious explanation — the `isSimulatedBySoftware` flag — is a **red herring** (Apple's own engineering states it does not fire for third-party/developer-channel tools), so the rejection must rest on a *different* check. Our differential isolates that check to one Pokémon GO performs and the controls do not, and our observations narrow it to a **motion-independent check evaluated at load** — it fires immediately and persists while the device is stationary, excluding trajectory, speed, and movement-based signals. We surface three candidates — network/location coherence, static sensor cross-validation, and a developer/instrumented-environment (delivery-method) fingerprint — and then **run the discriminating experiment**: spoofing to the device's *own* coherent location (matching its real IP, cell, and sensors). Error 12 **persisted, 3/3** — a coherent coordinate is *not* sufficient to clear it. This establishes that **coordinate content is not the discriminating variable** and undercuts the two content-based explanations (coherence, static sensor). It points *toward* the **delivery/method side** — that Pokémon GO rejects *how* the location is supplied rather than the coordinate — but cannot establish that conclusively, because a competing explanation survives the trial: the test account had prior spoof history, and heightened server-side scrutiny on a recently-teleported account would produce the same result. We therefore flag both an unresolved observability tension (how a sandboxed app observes the delivery environment — a GPS-stream fingerprint is the most plausible channel) and this account-state confound, which only a fresh, never-spoofed account could exclude. We therefore narrow — but do not conclusively pin — the mechanism, and are explicit about what remains uncertain. A survey of the 2025–2026 spoofing landscape shows that (a) all *software*/developer-mode tools share this limitation and are routinely broken by iOS/PoGo updates, and (b) the only currently-reliable no-jailbreak method is **MFi-certified external Bluetooth GPS hardware**, which presents to iOS as a legitimate GPS accessory and so avoids the trust gate — but is a hardware product, not reproducible in software (the barrier is Apple-signed cryptographic keys), and still not immune to ban-level detection. The tool is therefore reclassified as a **general-purpose iOS GPS spoofer** for apps without anti-cheat, and the paper's contribution is the documented, reproducible boundary between "the OS accepts a simulated fix" and "an anti-cheat-hardened app rejects it."

---

2. Introduction & motivation

Location-dependent applications increasingly need to be exercised at coordinates other than where the developer is sitting: navigation and mapping apps, geofenced features, fitness/route tracking, and location-gated content all behave differently by place. On Android this is a first-class capability — the OS exposes a "mock location" developer toggle that any app can read. **On iOS there is no user-facing mock-location API at all**, which makes host-side location control a genuinely interesting systems problem and the motivation for building "Phantom."

The project began with a deliberately demanding target: drive the spoof hard enough that **Pokémon GO** — a location game with a well-known, actively maintained anti-cheat — would accept it. Pokémon GO was chosen not as an end in itself but as the *hardest available test oracle*: if the strongest anti-cheat accepts a fix, weaker apps certainly will, and if it rejects one, the rejection is a precise, observable signal worth dissecting. The investigation therefore has two questions:

1. **Engineering:** can a non-jailbroken iPhone's GPS be controlled from a Windows host, reliably, in software? (Answer: yes.)
2. **Analysis:** at exactly what layer, and by what mechanism, does an anti-cheat-hardened app distinguish this from a real fix? (Answer: not where you'd first guess — see §9.)

The honest outcome is a **negative result for the original premise** (software cannot beat current PoGo anti-cheat) paired with a **positive, reusable artifact** (a working general-purpose iOS spoofer) and a precisely characterized detection boundary. Negative results with a clean root-cause analysis are the paper's intended contribution.

---

3. Background — how non-jailbreak iOS location spoofing works

Unlike Android (which exposes a "mock location" developer toggle any app can provide), iOS has **no user-facing mock-location API**. The only non-jailbreak mechanism is the **developer location-simulation service** that Xcode exposes as *"Simulate Location."* When set, every app — including Pokémon GO — reads it as the real GPS fix. The open-source library `pymobiledevice3` reimplements the protocols Xcode uses.

There are two eras:

iOS	Mechanism	Transport
<= 16	`com.apple.dt.simulatelocation` after mounting the DeveloperDiskImage (DDI)	usbmuxd
>= 17	Developer services moved behind a **RemoteXPC secure tunnel (RSD)**; location set via the DVT `LocationSimulation` instrument	RemoteXPC tunnel over USB

Our device is iOS 26.5, so the **iOS 17+ path** applies. The end-to-end chain is:

```
usbmuxd (Apple Mobile Device Service)  →  lockdown pairing/trust
   →  Developer Mode enabled on device
   →  RemoteXPC tunnel (tunneld) establishes an RSD endpoint
   →  DeveloperDiskImage (personalized, via Apple TSS) mounted
   →  DVT LocationSimulation.set(lat, lon)   ←  the actual spoof
```

Two non-obvious requirements that every working tool shares: - **Keep-alive:** a single `set()` is not enough. iOS lets a simulated fix decay, and on iOS 18+ it **resets when the USB cable is disconnected**. The fix must be re-sent continuously (~every 1.5 s) for the whole session, and the DVT instruments session must be held open. - **Jitter:** real GPS drifts a few metres constantly. A perfectly static coordinate is itself a detection signal, so each re-send is offset by ~1–5 m of random drift.

3.1 Location *provenance*: the concept the whole paper turns on

The key conceptual lever is that iOS does not merely report *where* a device is; since iOS 15 Core Location can also report *where the fix came from* — its **provenance** — via `CLLocationSourceInformation` (two booleans: `isSimulatedBySoftware`, `isProducedByAccessory`). The central analytical question of this paper is whether anti-cheat reads provenance directly, or infers it from other signals. §9 shows it is the latter — a finding that is not obvious and is, in fact, the opposite of the intuitive answer.

---

4. Threat model

Framing the problem as a small adversarial model clarifies what "success" and "detection" actually mean.

- **Defender:** Niantic/Scopely's anti-cheat, running partly in the Pokémon GO client and partly server-side, whose goal is to admit only *genuine* device locations and to flag or ban synthetic ones. It is assumed to be adaptive (models retrained over time) and to fuse multiple signals.
- **Attacker (this project):** a Windows host with USB access to a non-jailbroken, passcode-protected iPhone the author owns, using only Apple's own developer tooling (no jailbreak, no modified game binary). The attacker controls **GPS coordinates only** — not the device's inertial sensors, not its OS integrity state, not its network-derived location.
- **Assets / trust boundary:** the boundary of interest is **GPS source provenance**. Three provenance classes exist, in increasing order of trust: (i) software simulation via the developer/debug channel, (ii) an external MFi accessory, (iii) the genuine onboard GPS. The attacker can only produce class (i).
- **Capabilities the attacker lacks (and why they matter):** the inertial measurement unit (accelerometer/gyro/barometer/magnetometer) cannot be driven from the host, so any GPS motion the attacker injects is *uncorroborated* by the IMU; Developer Mode and an attached debug session are *present* on the device, though whether a sandboxed app can observe them is itself uncertain (§9.2); and the host cannot alter network/cell-derived location.
- **Success criterion (defined precisely for §8):** *the OS accepts and propagates the simulated fix to ordinary apps.* "Beating PoGo" is explicitly a **stretch** criterion, separated out because it depends on the defender, not the implementation.

This model predicts the result before we measure it: an attacker confined to provenance-class (i)

who cannot corroborate motion via the IMU should pass any app that ignores provenance and fail any app that fuses provenance + sensor signals. §8–§9 confirm exactly this.

---

5. Methodology

5.1 Apparatus

Component	Value
Host	Windows 11 Home (26200)
Device	iPhone 11 (`iPhone12,1`), iOS 26.5 (build 23F77)
Toolchain	Python 3.14, `pymobiledevice3` 9.30.1, Apple Mobile Device Service (from iTunes bundle)
Transport	USB; RemoteXPC tunnel via `pymobiledevice3 remote tunneld`
Spoof target	§8.2: 37.7749, −122.4194 (San Francisco), grossly incoherent with the device's true location. §8.4: ~330 m from the device's true position (Nairobi, Kenya), coherent with it.
Network egress	VPN **off**; §8.4 trial on cellular (Safaricom, Nairobi). Recorded per experiment, as it bears on coherence checks.

5.2 Procedure

1. Build the host-side spoofer (§6) and bring up the full chain of §3 to a verified "proof of life."
2. With the keep-alive loop holding a fixed target, observe the reported location in a set of independent consumer apps.
3. Record, for each app, whether it honors the simulated fix.

5.3 The controlled (differential) test — the core of the method

The decisive experiment is a **differential test across apps holding every other variable constant**. The same OS-level simulated fix, the same keep-alive loop, the same device state, were presented simultaneously to:

- **Apple Maps** (first-party, no anti-cheat) — *control*
- **Snapchat Map** (third-party, no location anti-cheat) — *control*
- **Pokémon GO** (third-party, hardened anti-cheat) — *treatment*
- **Independent variable:** the application reading the fix (specifically, whether it performs location anti-cheat).
- **Dependent variable:** whether the app accepts the location or rejects it (PoGo's "error 12").
- **Held constant:** the coordinate, the injection mechanism, permissions (While-Using + Precise ON), network, and device.

Because only the *app* varies, any difference in outcome is attributable to the app's own checks, not to the spoof's correctness. This is the experimental backbone of the paper and the basis for the §8 results.

5.4 Planned discriminating experiment — the "real-location" test

The differential (§5.3) proves PoGo does *something extra* but not *what* (§9.2 lists three surviving hypotheses: H1 network/location coherence, H2 static sensor cross-validation, H3 instrumented-environment fingerprint). A single further manipulation discriminates the paper's central *environment vs. location-content* question: **spoof to the device's own real location.**

Rationale. Pointing the spoof at where the phone physically is removes *every* location-content anomaly simultaneously — IP, cell/carrier region, barometric altitude, and magnetic field all become coherent with the reported GPS — **while leaving the instrumented environment unchanged** (Developer Mode on, DDI mounted, the DVT `LocationSimulation` channel still injecting). The outcome therefore splits the hypotheses:

Outcome of real-location test	Interpretation
Error 12 **still fires**	Location content is coherent yet it still fails → the trigger is the channel/environment → **H3 isolated**
Error 12 **clears**	The environment/channel is acceptable → the trigger was location-content → **H3 excluded; H1/H2 implicated**

Design refinement — offset, don't match exactly. The target is set ~300 m from the true position rather than exactly on it. At that distance coherence is preserved (same city/IP/cell/region; well within normal GPS+network error), but the offset is observable on a map — so a control app showing the dot at the offset point *verifies the spoof is actually engaged* rather than PoGo silently reading real GPS. An exact-match target would be unverifiable.

Controlled variables (additions to §5.1). Network egress is now a first-class variable because H1 depends on it: the test is run with **VPN off** so the IP is genuinely local and maximally coherent, and the egress configuration (VPN state, region, Wi-Fi vs cellular) is recorded for every trial. Account state is also recorded, since a prior teleport (e.g. the §8 San-Francisco test) may leave a residual soft-ban or server-side flag that confounds the result; any cooldown is waited out first.

Procedure. (i) Confirm clean account/cooldown state and VPN off. (ii) Read the device's true coordinates and offset one axis by ~300 m. (iii) Bring up Phantom identically to §5.3, changing only the target coordinate. (iv) Confirm in a control app (Apple Maps) that the offset point is shown — proving the channel is live. (v) Launch Pokémon GO, record outcome and time-to-result. (vi) Repeat >=3 trials, force-quitting between each.

Status. This experiment has now been **run**; results are in §8.4 and reorder the §9.2 hypotheses (it excluded content-coherence as a sufficient explanation). The per-trial data are in `TEST_RESULTS.md`. The test was non-destructive (no movement, no teleport, coherent fix) — the lowest-risk experiment available — subject to the inherent account/ToS risk noted in §10.

6. Implementation — what was built ("Phantom")

A two-part design per the original build plan: a Python device layer (the only place the protocols exist) behind a thin blocking API, intended to later sit under a web UI.

Device layer (`phantom/device.py`), delivered: - Runs one asyncio event loop in a background thread (a "bridge") and exposes blocking methods, because `pymobiledevice3` is fully async. - `connect()`: detects the device and iOS version, ensures `tunneld` is running, fetches the RSD endpoint, mounts the DDI (`auto_mount`), and opens a held-open `DvtProvider` + `LocationSimulation` session. - `set_location(lat, lon)` / `clear_location()`. - **Keep-alive loop:** re-sends the current target every 1.5 s with 1–5 m idle jitter (`phantom/geo.py: jitter_coord`), exactly matching the reference implementations. - Version branching: iOS 17+ via DVT/RSD (tested); iOS <=16 via `DtSimulateLocation` over usbmux (structured, untested — no legacy device available). - Clean teardown that clears the fix and restores real GPS.

The canonical verified command sequence (proof-of-life) was:

```
pymobiledevice3 usbmux list
pymobiledevice3 remote tunneld                                          # elevated
pymobiledevice3 mounter auto-mount --rsd <HOST> <PORT>
pymobiledevice3 developer dvt simulate-location set --rsd <HOST> <PORT> -- 37.7749 -122.4194
```

---

7. Engineering findings — every problem encountered and its fix

This is the practical heart of the paper: the friction of doing this on **Windows** (the build plan's "stretch target"), with root causes and fixes. Each is reported as symptom → root cause → fix so the work is reproducible.

7.1 Dependency compilation failure

- **Symptom:** `pip install pymobiledevice3` failed building `lzfse` and `pylzss` with *"Microsoft Visual C++ 14.0 or greater is required."*
- **Root cause:** Those two C-extension deps publish prebuilt Windows wheels only up to CPython 3.12. On Python 3.13/3.14 pip falls back to compiling from source, which needs a C toolchain.
- **Fix:** Installed Microsoft C++ Build Tools (MSVC v14.4) via `winget`; the deps then compiled and `pymobiledevice3 9.30.1` installed. (Alternative would have been Python <=3.12 to get wheels, but 3.14 was retained for its in-process userspace-tunnel support.)

7.2 No usbmux / "Failed to connect to usbmuxd socket"

- **Root cause:** Windows has no `usbmuxd` until Apple's **Apple Mobile Device Service** is installed.
- **Wrinkle:** Installing iTunes via `winget` reported success but **silently skipped the Apple Mobile Device Support sub-component** — no service, no `Common Files\Apple`.
- **Fix:** Re-downloaded the iTunes bundle, extracted `AppleMobileDeviceSupport64.msi` with 7-Zip, and installed it directly via `msiexec`. The service then ran and `usbmux list` detected the device.

7.3 Tunnel hangs (userspace) on Windows

- **Symptom:** `pymobiledevice3 lockdown start-tunnel --userspace` ran for minutes with **zero output and never established** a tunnel.
- **Root cause:** The pure-Python userspace tunnel path is unreliable/hangs on Windows.
- **Fix:** Switched to `pymobiledevice3 remote tunneld`, which established the RemoteXPC tunnel in ~5 s, exposes an HTTP API at `http://127.0.0.1:49151`, and **auto-recreates the tunnel after a device reboot**. (This is the same approach the working Windows reference implementations use.)

7.4 CLI crash when launched elevated: `sys.__stdout__` is None

- **Symptom:** Launching the tunnel elevated via redirected pipes crashed at import: `AttributeError: 'NoneType' object has no attribute 'fileno'` inside `blessed`/`inquirer3`.
- **Root cause:** When the process has no real console and stdout is a redirected pipe, `sys.__stdout__` is `None`; `blessed` calls `sys.__stdout__.fileno()` at import.
- **Fix:** Launch under a real console — `& cmd.exe /c "... 1> out 2> err"` — which keeps `sys.__stdout__` valid while still capturing output. (Commands run through a normal interactive shell are unaffected.)

7.5 Admin requirement even for "userspace" on Windows

- **Finding:** The CLI's `start-tunnel`/`tunneld` are gated by `@sudo_required`, so they demand elevation on Windows regardless of `--userspace`. The "no admin" promise of userspace mode only holds on macOS/Linux. Tunnel is therefore launched elevated via a UAC prompt.

7.6 Blind to elevated processes

- **Finding:** From a non-elevated shell, `Win32_Process.CommandLine` is **null** for elevated processes you don't own, so filtering tunnel processes by command line silently misses them. **Lesson:** check tunnel liveness via the `tunneld` HTTP API, not process scans.

7.7 Developer Mode would not appear (the biggest onboarding hurdle)

- **Symptom:** *"Developer Mode"* was absent from Settings → Privacy & Security; `mounter auto-mount` failed with *"Developer Mode is disabled."*
- **Dead ends:** `amfi enable-developer-mode` fails on any phone **with a passcode** (*"Cannot enable developer-mode when passcode is set"*), and *"Turn Passcode Off"* was **grayed out** (commonly caused by a paired Apple Watch, an Exchange mail policy, an MDM/config profile, or Screen Time restrictions). Establishing the tunnel, attempting a mount, and rebooting did **not** surface the toggle.
- **Fix:** `pymobiledevice3 amfi reveal-developer-mode` — a dedicated command that asks iOS to reveal the toggle. After running it, the toggle appeared; enabling it + rebooting set `developer-mode-status = true`, and the DDI then mounted (personalized image fetched via Apple's TSS server, ~2 min first time).

7.8 Proof of life

With all the above resolved, `simulate-location set 37.7749 -122.4194` moved the iPhone's blue dot to San Francisco in Apple Maps — feasibility of the **mechanism** confirmed.

8. Results

8.1 General spoof: success

The host-side spoofer works as designed. With the keep-alive loop running (re-send every 1.5 s + 1–5 m jitter), the device layer logged **zero keep-alive errors** over the session, and the simulated fix propagated to consumer apps system-wide.

8.2 The differential test: PoGo alone fails

App	Anti-cheat?	Result with identical OS-level fix
Apple Maps	No	Accepted — Shows San Francisco
Snapchat Map	No	Accepted — Shows San Francisco
Pokémon GO	Yes	Rejected — Briefly shows the avatar in SF, then **"Failed to detect location (12)"**

Interpretation. PoGo *received* the coordinate (the avatar appeared) before rejecting it, so the spoof reached the app correctly. Because two independent apps honored the *exact same* fix while only the anti-cheat-hardened app failed, the failure is attributable to **Pokémon GO's anti-cheat**, not to the implementation, permissions, or the fix itself. Per the threat model (§4), this is the predicted outcome for a provenance-class-(i) attacker.

8.3 What was tried before concluding it was anti-cheat (negative controls)

To rule out mundane causes, all of the following were attempted and **did not** resolve error 12: - Force-quitting and relaunching PoGo. - Toggling Location Services, Wi-Fi, and cellular off/on. - Verifying PoGo location permission = *While Using* with **Precise Location ON**. - Re-implementing the spoof with the continuous keep-alive loop so the fix was provably fresh and never frozen. - Considering an IP/GPS mismatch (active VPNs). (Initially set aside as a ban-risk signal; the §8.4 real-location trial later tested the coherence hypothesis directly — see below.)

8.4 The discriminating ("real-location") trial: coherent fix, still rejected

Per the protocol in §5.4, the spoof was set ~330 m from the device's *true* position (Nairobi, Kenya), making the reported GPS coherent with the device's real context — Safaricom cellular, Nairobi IP, matching altitude and magnetic field — with VPN off. Apple Maps showed the offset point, confirming the channel was live before each PoGo launch.

Trial	PoGo outcome	Time-to-result	Notes
1	**Error 12**	~3 s	Map briefly rendered at the offset, then error 12
2	**Error 12**	~3 s	Intermittent "going too fast / passenger?" speed popup while stationary (see below)
3	**Error 12**	~4 s	No speed popup

Result: error 12 persisted 3/3 at a coherent location. Contrast §8.2, where the coordinate was grossly *incoherent* (San Francisco GPS on a Nairobi device) and also failed. The identical outcome under both coherent and incoherent coordinates is the decisive data point: **a coherent coordinate is not sufficient to clear error 12** — coordinate content is not the discriminating variable.

The speed popup. The intermittent "you are going too fast" warning fired while the device was physically still. In normal play this trips above a modest, faster-than-running speed, so the most parsimonious cause is the ~330 m real→offset jump at session start being read as a brief velocity spike — a benign artifact of how the spoof engages, not evidence of a persistent movement anomaly. It appeared in only 1 of 3 trials and did not change the outcome.

What it establishes — and the competing explanation it cannot exclude. The robust conclusion is narrow and solid: a *coherent* coordinate was still rejected, so **coordinate content is not the discriminating variable.** What the trial does *not* settle is *why* the coherent fix failed, because two explanations survive it and it cannot distinguish them:

- **(a) Delivery-method / environment.** Pokémon GO rejects *how* the fix is supplied — the developer/instrumented environment or the shape of the injected GPS stream — independent of the coordinate (§9.2, H3).
- **(b) Server-side account state.** The account had teleported to San Francisco the prior day. Leaving location off for 12 h clears Niantic's *cooldown* timer, but **cannot be verified to have reset any server-side account flag.** A recently-teleported account may be placed under heightened scrutiny that rejects subsequent location submissions *regardless of their coherence* — which would produce exactly this result with no reference to the delivery method at all.

We initially leaned toward (a) and judged (b) low-probability, but we are not in a position to assert that: we have **no visibility into Niantic's server-side logic** (the black-box-defender limitation, §11), so we cannot claim to know what "shape" a flagged account's rejection takes. (a) and (b) are **confounded in this trial**, because the only account available had prior spoof history. Cleanly separating them requires re-running this experiment on a **fresh, never-spoofed account** (ideally alongside a clean control account), which we did not do. We therefore report (a) as **consistent with — not established by** — this trial, and carry the account-state confound forward explicitly (§9.2, §11).

---

9. Analysis — the trust gate, and a debunked easy answer

> **Epistemic note.** The signals below are stated at differing confidence levels. We mark each: [**Confirmed**] = directly observed in this study or documented by Apple; [**Inferred**] = strongly implied by our threat model and behavior but not directly measured here; [**Reported**] = asserted by secondary/grey-literature sources (commercial spoofing vendors) and treated as market signal, not technical authority. This labeling is deliberate: much public writing on PoGo anti-cheat is vendor marketing, and a credible analysis must separate what it *knows* from what it *infers*.

PoGo's rejection is almost certainly produced by **cross-signal corroboration**: no single check, but several signals fused into anomaly models retrained over time. Our differential test (§8.2) establishes one thing cleanly and *only* that one thing: **Pokémon GO performs at least one location-validation check on load that Apple Maps and Snapchat do not.** It does **not**, on its own, tell us *which* check fires — that question is the subject of §9.2, and we are careful below to separate what the differential proves from what it merely permits.

Candidate signal layers (confidence-marked individually):

1. **Network / location coherence** — cell-, Wi-Fi-, and IP-derived position vs reported GPS. Motion-independent; checkable the instant the app loads. [**demoted by §8.4 — a coherent fix still failed**]
2. **Sensor cross-validation** — GPS vs accelerometer/gyro/barometer/magnetometer. Trajectory mismatches (moving GPS, static IMU) need motion to trip; *static* mismatches (barometric altitude, magnetometer) do not. [**demoted by §8.4 — sensors matched the reported place, yet it failed**]
3. **GPS source provenance** — trusted source vs developer/simulation channel. The intuitive gate — but see §9.1 for why the obvious form of this is a red herring. [**Inferred**]
4. **Movement / speed analysis** — teleports, impossible speeds; the "cooldown" basis. Requires motion. [**Reported**]
5. **GPS jitter fingerprinting** — perfectly static or perfectly linear coordinates look synthetic. [**Inferred**]
6. **Instrumented/developer-environment tells (incl. GPS-stream fingerprint)** — Developer Mode on, DDI mounted, live RemoteXPC/DTX session, and the statistical shape of the injected fix stream. [**best-supported after §8.4 — but the observability of the device-state tells remains the open puzzle (§9.2 H3)**]
7. **Behavioral models** — grid-walking, 24/7 activity, instant reactions. [**Reported**]

Enforcement is a 3-strike escalation: warning → ~7-day suspension → permanent ban; breaking the movement "cooldown" yields a ~12-hour soft ban. [**Reported**]

9.1 A tempting wrong answer: `CLLocationSourceInformation`

It is tempting to assume error 12 is simply an app reading a public "is this fake?" flag. Since **iOS 15**, every `CLLocation` can carry a `sourceInformation` property (`CLLocationSourceInformation`) with two booleans:

```
location.sourceInformation?.isSimulatedBySoftware    // software-simulated location
location.sourceInformation?.isProducedByAccessory    // location from an external accessory
```

At first glance this looks like *the* mechanism. **It is not — and this is a result worth stating,**

because the obvious explanation is wrong. Per an Apple DTS engineer on the developer forums, `isSimulatedBySoftware` is set **only when location is simulated through Core Location's own integration — i.e. Xcode's debugger loading a GPX file.** A third-party tool acting as an external location provider — *including the developer DVT `simulate-location` path that Phantom and the commercial tools use* — does **not** get this flag set: [**Confirmed — Apple DTS**]

> "Any third-party tool can be utilized as an external location provider, but it will not have access to set that API flag. Only when simulating with Core Location will the flag be set to true using the Xcode debugger and loading in Xcode a GPX file." — Apple DTS, developer forums thread 803179

The practical consequence: an app that naïvely checks `isSimulatedBySoftware` would **fail to catch** our spoof. Therefore PoGo's error 12 **cannot** be that single check — it must rest on some *other* signal, and §9.2 narrows the candidates without claiming to identify the one that fires. This is the paper's most defensible technical finding: the easy API is a red herring. `isProducedByAccessory`, by contrast, *is* set for genuine MFi accessories — which §9.4 examines.

The conclusion of §9.1 — that an app naïvely reading `isSimulatedBySoftware` would *not* catch our spoof — rests on a single primary source (Apple DTS, forum thread 803179) and our threat-model reasoning, not on direct measurement in this study. It is, however, **cheaply confirmable, and we package the means to do so**: `sourceinfo-probe/` in the project repository is a minimal first-party SwiftUI app that reads `location.sourceInformation` live and displays `isSimulatedBySoftware` / `isProducedByAccessory` while our DVT keep-alive runs. Executing it requires a macOS/Xcode build-and-sign toolchain we did not have access to, so the measurement is left as a **ready-to-run, reproducible artifact** rather than a result reported here: anyone with a Mac and an iOS 15+ device can confirm or falsify the claim in minutes (expected: `isSimulatedBySoftware = false` under the developer channel; a positive Xcode-GPX control should read `true`, demonstrating the Apple DTS distinction on a single device). The claim therefore remains [**Reported — Apple DTS**] rather than [**Confirmed by us**] — but the exact falsifiable test and its code ship with the paper, which we consider a stronger position than an untestable assertion. *(Build/run/record protocol: `sourceinfo-probe/README.md`; see also §11.)*

9.2 Which check fires? Three live hypotheses, ranked by what a sandboxed app can actually observe

The differential (§8.2) tells us PoGo does *something extra*. It does not tell us *what*. Three hypotheses survive our observations; we rank them not by narrative appeal but by a concrete constraint that an earlier draft of this analysis overlooked: **Pokémon GO runs inside the iOS application sandbox, so it can only act on signals it can actually read from there.** A hypothesis that requires reading a device state for which no sandbox-available channel is known is, absent evidence of such a channel, *weaker*, not stronger.

What our key observations actually constrain: - **Maps & Snapchat accept the identical fix** → the coordinate is valid/well-formed; error 12 is not "bad/no location." Rules out malformed-fix explanations only. - **It fires immediately and persists while stationary** → rules out checks that *require motion* (trajectory mismatch, speed, jitter-over-time). It does **not** rule out motion-independent checks (network coherence, static sensor mismatch, environment tells). - **It is not `isSimulatedBySoftware`** (§9.1) → rules out the naïve provenance-flag read in its obvious form.

So the field narrows to **motion-independent, load-time** checks. Three remain. First, what a sandboxed app can actually read:

Signal	Readable by a sandboxed app?	Set/violated by our setup?
`isSimulatedBySoftware`	Yes (`CLLocation.sourceInformation`)	**No** (§9.1) — excluded
Network/Wi-Fi/cell-derived position	IP via any request (independent); cell region via CoreTelephony. NB a *Core Location* network fix is itself overridden by the simulation.	Incoherent in the §8.2 SF run; **coherent** in the §8.4 Nairobi run — **yet both failed**
IMU: barometer, magnetometer (static)	Yes (Core Motion)	Plausibly — altitude/magnetic field won't match SF
GPS jitter pattern	Yes (sample the fix stream)	Only if our jitter looks synthetic; needs samples
Developer Mode status	**No known public/sandbox API**	State is on, but likely unobservable from app
DDI mounted	**No known public/sandbox API**	State exists, likely unobservable from app
`tunneld`/RemoteXPC/debugserver running	**No** — iOS apps cannot enumerate other processes	Daemons exist, but not visible to a sandboxed app
`P_TRACED` on PoGo's own process	Yes (`sysctl KERN_PROC`)	**No** — DVT `LocationSimulation` does not `ptrace`-attach to PoGo

The table narrows the field to motion-independent, sandbox-readable signals. The real-location trial (§8.4) then tested the content-based candidates **directly**, and the result reorders everything.

The discriminating result (§8.4). We ran the experiment this section originally only proposed — spoofing to the device's *own* coherent location (right city, IP, cell, and sensors; ~330 m offset):

Spoof to a coherent real location →	H1 (coherence)	H2 (static sensor)	H3 (environment/method)
Predicted error 12	clears	clears	still fires
Observed	—	—	**still fires (3/3)**

Error 12 **persisted at a coherent coordinate.** Both content-based hypotheses predicted it would clear; it did not. The incoherent SF run (§8.2) and the coherent Nairobi run (§8.4) produced the *same* failure — so coordinate content is not the discriminating variable.

H1 — Network/location coherence. **[demoted — undercut by §8.4]** If instantaneous IP/cell/sensor coherence were the gate, the coherent Nairobi run should have passed; it failed. So coherence is not the sole trigger. It may still contribute as one of several fused signals, but it is no longer the leading single explanation — and the earlier draft's "coherence is the prime suspect" is retracted.

H2 — Static sensor cross-validation. [**demoted — undercut by §8.4**] In Nairobi the barometric altitude and magnetic field *matched* the reported place, yet error 12 still fired. A static-sensor *mismatch* cannot explain a failure that occurs when the sensors agree.

H3 — Developer/instrumented-environment (delivery-method) fingerprint. [**promoted — best-supported, with an open puzzle**] With content coherent and still rejected, the trigger is *how the fix is delivered*, not the coordinate — the H3 class. This is now the best-supported explanation, but §8.4 *sharpens* rather than closes the observability tension: if PoGo rejects the method, by what sandbox channel does it observe it? We still know of no API exposing Developer Mode, a mounted DDI, or the `tunneld`/debugserver daemons. Two possibilities keep H3 viable: - **A GPS-stream fingerprint** — the keep-alive re-sends a jittered fix every 1.5 s; the *statistical shape* of that stream (cadence, jitter distribution, absence of genuine sensor-fused noise) is readable by sampling `CLLocationManager`, and would flag the *method* without reading any device-state flag. This is the most observable member of the method class and may be the actual channel. - **Undocumented / side-channel environment checks** a hardened anti-cheat may employ that we cannot enumerate.

So "delivery-method, not content" is well-supported; "specifically Developer-Mode/DDI detection" is one mechanism within it and — on observability grounds — arguably *not* the most plausible. The GPS-stream fingerprint is the more sandbox-realistic channel.

A precision note on `P_TRACED`. It is tempting to attribute detection to the classic iOS anti-debug check — the `P_TRACED` flag read via `sysctl(KERN_PROC)`. But that flag is set only when a debugger `ptrace`-attaches to *the app's own process*, and DVT `LocationSimulation` is a device-level instrument that does **not** attach to PoGo. So `P_TRACED`-on-self is most likely not set by our method and is probably not load-bearing. It remains a plausible *secondary* signal but should not be assumed to trigger. [**Inferred**]

Residual confound — a genuine competing hypothesis, not a footnote. The account had spoofed to San Francisco the previous day. Leaving location off for 12 h clears Niantic's *cooldown* timer, but we cannot verify it reset any server-side account flag. We cannot rank this "low-probability": with no view into Niantic's server logic, we have no basis to assert what form a flagged account's rejection takes. So "**heightened scrutiny on a recently-teleported account**" stands as a full alternative to the delivery-method explanation — it predicts error 12 at a coherent location just as well — and §8.4 cannot exclude it because the only test account had prior spoof history. Only a fresh, never-spoofed account (§11) could separate the two.

Honest confidence. §8.4 supports exactly one strong claim: it *excludes instantaneous content-coherence as a sufficient explanation* (a coherent fix was rejected 3/3), so **coordinate content is not the discriminating variable.** It does **not** by itself establish "delivery-method," because the server-side account-state hypothesis above survives the same trial. *Conditional* on the confound being benign, the delivery/method side is the better-supported reading, and within it the developer/instrumented-environment class (H3) — most plausibly via a **GPS-stream fingerprint** — is the leading channel. But that conditional is load-bearing: we have **not** pinned the mechanism, and the server-side-history confound is a live competitor, not a residual caveat. The earlier "network coherence is the prime suspect" framing is retracted in light of §8.4. [**Inferred — strong for "content is not the discriminator"; weaker, and confound-dependent, for "method, not account-state"**]

9.3 Why our method fails: the two classes of no-jailbreak spoofing

Class	How it works	PoGo error 12?	Buildable in software?
A. Developer/DVT `simulate-location` (our tool; also iAnyGo, iMyFone AnyTo, 3uTools over USB)	Drives Apple's developer debugging protocol to inject coordinates	**Yes — detected.** Developer Mode + instruments session + the injected fix-stream shape are candidate signals (§9.2)	Yes (this is what we did)
B. MFi external Bluetooth GPS (Brook Flashman, iTools BT 2.5)	A Bluetooth accessory with Apple's **MFi auth chip** presents as a *real external GPS receiver*; iOS Core Location treats it as a trusted hardware source	**No — accepted.** Looks like an ordinary GPS dongle, no developer-mode/debug signal	**No** — requires the MFi authentication chip; it is hardware
(C. Modified IPA clients — out of scope)	Repackaged game binary	Detected at login via client fingerprint	n/a

The critical asymmetry between class A (detected) and class B (accepted) is often *attributed* to provenance/environment differences — but we must be candid that **we measured only class A.** Our class-B claim ("MFi Bluetooth GPS is accepted") rests entirely on vendor/grey-literature sources (Brook Flashman, iTools BT) marked [**Reported**]; we did not test such hardware. An earlier draft called the A-vs-B difference "strong corroboration" for the instrumented-environment thesis. We retract that strength: comparing a measured result against untested marketing cannot corroborate a mechanism. The asymmetry is *suggestive* — class B plausibly avoids Developer Mode, DDI, and any debug session, and *also* presents a coherent hardware fix — but because it differs from class A on **multiple axes at once** (environment *and* provenance *and* potentially coherence), it cannot isolate which axis matters. [**Reported / Inferred — not corroborated by direct test.**]

The robust, measurement-backed claim is narrower and still worth stating: **no amount of better code moves class A into class B**, because the missing pieces (a trusted hardware source; absence of any developer-channel state) are not code defects. That conclusion stands regardless of which of H1–H3 is the actual trigger.

Furthermore, even class B only spoofs **coordinates**, not sensors — so it clears the *entry* gate (error 12) but remains exposed to the ban-level checks (#2–#7 above). **"Getting in" is not "being undetectable."**

9.4 Why a laptop's own Bluetooth radio cannot impersonate class B

A natural follow-up: *if class B works, can a laptop's Bluetooth chip simply present itself to the iPhone as a GPS accessory?* The answer is **no**, for two independent reasons:

1. **The MFi authentication handshake.** For iOS to treat a Bluetooth device as a *location source* (the path that sets `isProducedByAccessory`), the device must connect through Apple's **MFi External Accessory** programme, which performs a **cryptographic challenge-**

response at pairing time: the iPhone requests the accessory's Apple-signed certificate, then issues a challenge that only the accessory's **authentication coprocessor** can sign (RSA-1024/SHA-1 in the older MFi authentication coprocessor generation; newer MFi auth ICs use ECC-based challenge-response — private key held in Apple-provisioned silicon either way). A commodity laptop Bluetooth radio can speak the wire protocols but cannot answer the challenge — the required private keys live only inside genuine Apple-issued chips. This is a cryptographic barrier (analogous to forging a TLS certificate without the CA's private key), not a missing-API or undocumented-protocol barrier. [**Confirmed — Apple MFi security documentation + iAP2 teardown**]

2. **Real accessories report real fixes.** Even setting the handshake aside, standard MFi Bluetooth GPS receivers (Bad Elf, Garmin GLO, Dual XGPS) compute a fix from actual satellites and report *that* — they expose no coordinate-injection input. So even a genuine, certified dongle cannot be made to lie about position.

This is exactly why the **Brook Flashman** is a hardware *product* rather than a software download: it is the unusual device that combines *both* a valid MFi auth chip *and* a coordinate-injection input. That combination — not unwritten code — is the missing piece. The accessory path is therefore unreachable from a software-only project on a host PC, from a second independent direction (the auth keys) beyond the cost/ban arguments.

9.5 The software tools are themselves broken on current iOS

The commercial *software* tools (iAnyGo etc.) are the **same class A method**. They survive only by frequent cat-and-mouse updates, and there are active vendor guides titled *"iAnyGo not working on iOS 26"* — corroborating that the developer/DVT approach is broadly detected on the latest iOS, not merely in our hands. The vendor "fixes" for error 12 (enable game mode, toggle location, relaunch) are the same band-aids tried here and do not address the root cause. Independent reporting traces the current wave of error 12 to a Niantic anti-spoofing/GPS-tracking change first widely observed around mid-2024 and reinforced by subsequent client updates. [**Reported**]

---

10. Ethics & legal considerations

This project touches a live, commercially operated online game, so it was conducted under explicit constraints:

- **Own device, own account, no third-party harm.** All testing used the author's own iPhone and account. No other players were targeted, no game economy was manipulated for gain, and no production service was load-tested or attacked. The PoGo interaction was limited to observing whether the client accepts a simulated fix on launch.
- **Terms of Service.** Spoofing location in Pokémon GO violates Niantic's Terms of Service and carries a documented escalating ban risk (§9). This paper does **not** advocate or instruct circumventing that anti-cheat; in fact its central finding is that doing so is not achievable by the software method studied, and it deliberately stops at characterizing *why* rather than seeking a bypass.
- **No anti-cheat evasion technique is provided.** The analysis explains the detection boundary (provenance, sensor fusion, MFi cryptography) at the conceptual level needed to understand the negative result. It does not develop IMU-spoofing, debug-signal hiding, MFi-key

extraction, or modified clients — those were treated as out of scope precisely because they cross from "understanding the boundary" into "defeating a third party's protections."

- **Reframing to legitimate use.** The reusable artifact is positioned for legitimate location testing (navigation/geofence/fitness QA, location-gated feature testing) on apps that do not implement anti-cheat — the use case the OS mechanism is actually intended for.
- **Responsible-disclosure posture.** No vulnerability in Apple or Niantic systems was discovered or exploited; the work uses only Apple's documented developer tooling. There is therefore nothing to disclose, but the standard would have been to report rather than publish a working bypass.

---

11. Limitations & future work

Limitations: - **Single device / single OS.** Results are from one iPhone 11 on iOS 26.5. The iOS <=16 `DtSimulateLocation` path is implemented but **untested** (no legacy device available). - **Black-box defender.** The §9 anti-cheat signal list is partly inferred and partly grey-literature; we observe *that* PoGo rejects the fix and can bound *why*, but cannot see Niantic's server-side logic. The epistemic markers in §9 are the honest expression of this. - **No quantitative detection study.** The result is a clean qualitative differential (accept vs reject), not a measurement of, e.g., how long a fix survives or how speed thresholds map to soft-bans. - **Time-bound.** Anti-cheat and iOS both move; specifics (error code, tool versions, the `isSimulatedBySoftware` behavior) are accurate as of June 2026 and may drift. - **Server-side account-state confound (the main threat to §8.4's reading).** The §8.4 trial used an account that had spoofed to San Francisco the prior day; the 12 h location-off wait clears Niantic's *cooldown* but not, verifiably, any account *flag*. "Heightened scrutiny on a recently-teleported account" predicts error 12 at a coherent location just as well as the delivery-method explanation does, and this trial cannot tell the two apart. We do not rank it low-probability (we cannot see the server logic). The clean fix — re-running §8.4 on a fresh, never-spoofed account plus a clean control — is the highest-value follow-up to this paper.

Future work (legitimate directions): - **(Done) Discriminating experiment — §8.4.** Spoofing to the device's own coherent location was run; error 12 persisted 3/3, excluding content-coherence as a sufficient explanation and shifting weight to the delivery-method class. The remaining open question is the *specific* channel: a targeted follow-up would test the **GPS-stream-fingerprint** hypothesis — e.g. vary the keep-alive cadence and jitter distribution and observe whether error 12 tracks the stream's statistics rather than any device-state flag. - **Broaden the app matrix.** Test more anti-cheat-free apps (fitness, navigation, dating, enterprise geofencing) to map which categories read provenance — turning the single differential into a survey. - **Build a "compatibility self-test"** into Phantom: confirm OS-level acceptance and clearly report the known limitation for anti-cheat apps, rather than failing silently. - **Harden the device layer:** reconnect/teardown robustness, multi-device support, and validating the iOS <=16 path on a legacy device. - **Execute the bundled provenance probe (built, ready to run).** `sourceinfo-probe/` is a complete minimal app that measures `isSimulatedBySoftware` under the DVT/developer channel; it needs only a macOS/Xcode toolchain (unavailable to us) to build, sign, and deploy. Running it — with the Xcode-GPX positive control — would upgrade §9.1 from [**Reported — Apple DTS**] to [**Confirmed by us**] in minutes. Full build/run/record protocol: `sourceinfo-probe/README.md`. A natural way to get it run is to share the probe in the `pymobiledevice3` community, for whom the result is directly useful. - **Provenance documentation contribution.** The `isSimulatedBySoftware` "red herring" finding (§9.1) is under-

documented publicly and worth writing up on its own.

---

12. Conclusion & recommendations

1. **The original premise is not viable.** Building "an iAnyGo-equivalent in software that works with Pokémon GO" fails because the software approach *is* iAnyGo's approach, and that approach no longer reliably beats current PoGo anti-cheat on modern iOS. The block is **structural, not an implementation gap**: §8.4 shows that even supplying a *coherent* coordinate (right city, IP, cell, sensors) does not clear error 12, so the missing property is not a better coordinate but a trusted *delivery method* — which a host-side injector cannot provide, and no amount of better code changes that. We locate the load-bearing property on the delivery-method side (§8.4) but do not pin the exact channel (§9.2).
2. **The block is method, not coordinate — and we say only what we can defend.** Error 12 is *not* a simple `isSimulatedBySoftware` read: that flag does not fire for developer-channel tools, so any app relying on it would miss our spoof. This **red-herring result is the paper's sharpest defensible finding** (§9.1), independently confirmable with a small test app. Beyond it, §8.4 is decisive on one axis only: a *coherent* coordinate was still rejected 3/3, so **the coordinate itself is not the discriminating variable** — which excludes the content-based explanations (coherence, static sensor). It points *toward* the delivery-method / instrumented-environment class (most plausibly a **GPS-stream fingerprint**), but does not establish it: a competing explanation — heightened server-side scrutiny on an account with prior spoof history — survives the trial and would produce the same result. Separating the two needs a fresh, never-spoofed account (§11). The earlier "network coherence is the prime suspect" framing is retracted in light of §8.4.
3. **Reclassify the project** as a general-purpose, non-jailbreak iOS GPS spoofer for apps without anti-cheat. Continue the movement engine and UI for that use; **drop the Pokémon-GO-specific cooldown subsystem**, which only applies if PoGo were playable.
4. **For Pokémon GO specifically,** the only currently-reliable no-jailbreak route is **MFi Bluetooth GPS hardware** — a product, not a build target — and it reduces, not eliminates, ban risk. It is out of scope on both technical (Apple-signed keys) and ethical (ToS) grounds.
5. **The key takeaway is the boundary itself:** the spoof "works" (Maps/Snapchat prove it); the anti-cheat-hardened *game* refuses it, by design, for **structural reasons** a host-side tool cannot address. Documenting that boundary precisely is the contribution.

---

13. References

Primary / authoritative - pymobiledevice3 — https://github.com/doronz88/pymobiledevice3 - iOS 17+ tunnels guide — https://github.com/doronz88/pymobiledevice3/blob/master/docs/guides/ios17-tunnels.md - pymobiledevice3 protocol layers (RemoteXPC / RSD / DDI) — https://github.com/doronz88/pymobi - Apple — `CLLocationSourceInformation.isProducedByAccessory` (Core Location) — https://developer.apple.com/documentation/corelocation/cllocationsourceinformation/isproducedbyaccessory - Apple DTS — `isSimulatedBySoftware` does not flag third-party/external providers (forum thread 803179) — https://developer.apple.com/forums/thread/803179 - Apple Developer Forums — "Location Spoofing Detection" (isSimulatedBySoftware usage, iOS 15+) — https://developer.apple.com/forums/thread/120491 - Apple Support — Verifying

accessories for iPhone and iPad (MFi authentication) — https://support.apple.com/en-ie/guide/security/sec70a4f377d/web - MFi iAP2 challenge-response / authentication coprocessor (RSA-1024/SHA-1) teardown — https://wiomoc.de/misc/posts/mfi_iap.html - Reference implementation (DVT method) — https://github.com/gabrielvuksani/iphonespoofer

Grey literature (commercial / market signal — treated as Reported, not authoritative) - iAnyGo — Pokémon GO error 12 guide — https://www.ianygo.com/change-location/pokemon-go-failed-to-detect-location-12.html - iAnyGo "not working on iOS 26" — https://www.ianygo.com/change-location/ianygo-not-working.html - iMyFone AnyTo — error 12 — https://anyto.imyfone.com/pokemon-go/pokemon-go-error-12/ - iFlowGo — iOS spoofing mechanism breakdown (DVT vs Bluetooth vs MFi; error 12 = sensor mismatch) — https://iflowgo.com/pokemon-go-ios-spoofer-full-review/ - Brook Flashman (MFi Bluetooth location controller) — https://store.brookgaming.com/products/flashman - iTools BT 2.5 dongle review — https://www.locachange.com/pokemon-go/itools-dongle-mobile-bt-device-pokemon-go/ - Sulpog (ESP32 — Go Plus auto-catcher, *not* a location spoofer) — https://github.com/tristannottelman/Sulpog - Cooldown chart (PoGo soft-ban rules) — https://www.pgsharp.com/cooldown-rules/

Full engineering detail and the reproducible command sequences are in §6–§7; the host-side device layer is the `phantom/` *package in this repository.*